

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL 3 – INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	V.Sireesha
Register Number	192472208

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2

Total Marks: 20

Code of Conduct:

I V.Sireesha (192472208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

V.Sireesha

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

the performance and list at least two misclassified instances.

1.(a) Load, Pre-process, and Split the Dataset

Program:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print('First 5 rows:')
print(df.head())

print('\nData Types:')
print(df.dtypes)

print('\nMissing Values:')
print(df.isnull().sum())

# Preprocessing note: the Iris dataset has no missing values and all #
# features are already numeric, so no imputation or encoding is required.

X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print('\nTraining samples:', len(X_train))
print('Testing samples:', len(X_test))
```

Output:

First 5 rows: sepal length (cm) sepal width (cm) ... petal width

```
(cm) target
0      5.1    3.5 ... 0.2    0 1    4.9    3.0
...    0.2    0 2    4.7    3.2 ... 0.2    0 3
      4.6    3.1 ... 0.2    0
4      5.0          3.6 ...          0.2    0
```

[5 rows x 5 columns]

Data Types: sepal length

(cm) float64 sepal width

(cm) float64 petal length

(cm) float64 petal width

(cm) float64

Target int64

dtype: object

Missing Values:

sepal length

(cm) 0 sepal width

(cm) 0 petal length

(cm) 0 petal width

(cm) 0 target 0

dtype: int64

Training samples: 120

Testing samples: 30

(b) Build the Decision Tree (Gini Index) and Identify the Root

Node

Program:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
```

```
model = DecisionTreeClassifier(criterion='gini', random_state=42, max_depth=4)
model.fit(X_train, y_train)
```

```
print('Tree Depth:', model.get_depth())
print('Number of Leaves:', model.get_n_leaves())
```

```
print('\nFeature Importance:') for feature, importance in zip(iris.feature_names,
model.feature_importances_): print(f' {feature}: {importance:.4f}')
```

```
root_index = model.tree_.feature[0] root_feature =
iris.feature_names[root_index] root_threshold =
model.tree_.threshold[0] print(f'\nRoot Node: {root_feature}
(threshold = {root_threshold:.2f})')
```

```
plt.figure(figsize=(16, 9))
plot_tree(model, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True, fontsize=8)
plt.title('Decision Tree - Iris Dataset')
plt.show()
```

Output:

Tree Depth: 4

Number of Leaves: 7

Feature Importance:

sepal length (cm):

0.0063 sepal width

(cm): 0.0169 petal

length (cm): 0.5656

petal width (cm):

0.4112 Root Node:

petal length (cm)

(threshold = 2.45)

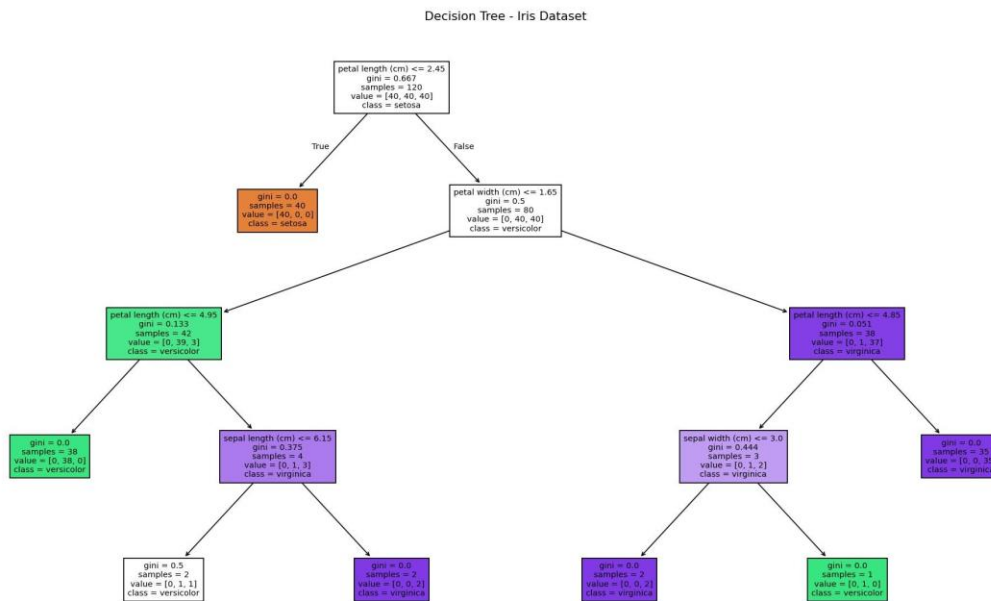
Decision Tree Structure (text form):

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|       |--- petal length (cm) <= 4.95
|       |   |--- class: 1
|       |   |--- petal length (cm) > 4.95
|       |       |--- sepal length (cm) <= 6.15
|       |       |   |--- class: 1
|       |       |   |--- sepal length (cm) > 6.15
|       |       |       |--- class: 2
|       |--- petal width (cm) > 1.65
```

```

| | |--- petal length (cm) <= 4.85
| | |--- sepal width (cm) <= 3.00
| | |--- class: 2
| | |--- sepal width (cm) > 3.00
| | |--- class: 1
| | |--- petal length (cm) > 4.85
| | |--- class: 2

```



Root Node Identification & Justification:

The root node splits on 'petal length (cm)' at a threshold of 2.45 cm. This feature is chosen automatically by the CART algorithm because, among all four features, splitting on petal length at this threshold produces the largest reduction in Gini impurity — it perfectly separates all 40 Setosa training samples (Gini = 0.0 in the left child) from the other two classes in a single split. The feature-importance scores confirm this numerically: petal length (≈ 0.57) and petal width (≈ 0.41) together account for over 97% of the total importance, while sepal length and sepal width contribute almost nothing, so the tree correctly prioritizes petal dimensions — which biologically separate Iris species far more cleanly than sepal dimensions.

(c) Evaluate the Model: Accuracy, Confusion Matrix, Classification Report

Program:

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)

cm = confusion_matrix(y_test, y_pred)
print('\nConfusion Matrix:')
print(cm)

print('\nClassification Report:') print(classification_report(y_test, y_pred,
target_names=iris.target_names))

mask = y_test.values != y_pred
misclassified =
pd.DataFrame(X_test.values[mask], columns=iris.feature_names)
misclassified['Actual'] = [iris.target_names[i] for i in y_test.values[mask]]
misclassified['Predicted'] = [iris.target_names[i] for i in y_pred[mask]]
print('\nMisclassified Instances:') print(misclassified)
```

Output:

Accuracy: 0.9333

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 1]
 [ 0 1 9]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
Accuracy		0.93	0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Misclassified Instances: sepal length (cm) sepal width (cm) ...

	Actual	Predicted		
0	6.1	2.6 ...	virginica	versicolor
1	6.7	3.0 ...	versicolor	virginica

[2 rows x 6 columns]

Number of misclassified instances: 2 out of 30

Comment on Performance:

The model achieves 93.33% accuracy on the held-out test set (28 out of 30 correctly classified). The confusion matrix shows all 10 Setosa samples classified perfectly (this class is linearly separable from the other two, as seen in the tree's very first split). The two errors both occur between Versicolor and Virginica — the classification report confirms this with slightly lower precision/recall (0.90) for exactly these two classes, which is expected since their petal measurements naturally overlap at the boundary. The two misclassified instances are:

- Instance 1 — sepal length 6.1, sepal width 2.6, petal length 5.6, petal width 1.4: Actual = virginica, Predicted = versicolor.
- Instance 2 — sepal length 6.7, sepal width 3.0, petal length 5.0, petal width 1.7: Actual = versicolor, Predicted = virginica.

Both misclassifications happen exactly at the ambiguous petal-length/petal-width boundary the tree splits on near 4.85–4.95 cm — these are borderline flowers whose measurements sit close to the decision threshold, which is the natural limit of a single decision tree's precision on overlapping classes.

2.(a) Define the Environment and Initialize the Q-Table

The environment is a 4×4 grid (16 states) with a start state at (0,0) and a goal state at (3,3). Obstacles are placed at (1,1) and (2,2). The agent can move Up, Down, Left, or Right; moving into a wall keeps the agent in place with a -1 penalty, moving into an obstacle gives 5, reaching the goal gives +10, and every other normal step gives -1. The Q-table is a 4×4×4 array (state row, state column, action) initialized to all zeros.

Program:

```
import numpy as np
import random
```

```
N = 4
```

```
start, goal = (0, 0), (3, 3)
```

```
obstacles = {(1, 1), (2, 2)}
```

```
moves = [(-1,0), (1,0), (0,-1), (0,1)] # Up, Down, Left, Right
```

```
action_names = ['Up', 'Down', 'Left', 'Right']
```

```

# Hyperparameters
alpha, gamma = 0.1, 0.9      # learning rate, discount factor
epsilon, decay, eps_min = 1.0, 0.98, 0.05 # epsilon-greedy exploration, decayed each episode
episodes, max_steps = 100, 100

# Q-table: shape (4, 4, 4) -> (row, col, action)
Q = np.zeros((N, N, 4))

def step(state, action):
    r, c = state
    nr, nc = r + moves[action][0], c + moves[action][1]
    if not (0 <= nr < N and 0 <= nc < N):
        return state, -1, False      # wall bump
    ns = (nr, nc)
    if ns in obstacles: return state, -5, False      # obstacle penalty
    if ns == goal:
        return ns, 10, True          # goal reached
    return ns, -1, False             # normal step

```

Hyper parameters used:

- Learning rate (α) = 0.1
- Discount factor (γ) = 0.9
- Exploration rate (ϵ) = starts at 1.0, decays by $\times 0.98$ each episode down to a floor of 0.05 (epsilon-greedy with decay, so the agent explores heavily early and exploits its learned policy later)

(b) Run Q-Learning for 100 Episodes — Q-Table Evolution

Program:

```

for ep in range(1, episodes + 1):
    state = start
    total_reward = 0
    for _ in range(max_steps):
        if random.random() < epsilon:
            action = random.randrange(4)      # explore else:
            action = int(np.argmax(Q[state]))  # exploit
        next_state, reward, done = step(state, action)
        target = reward if done else reward + gamma * np.max(Q[next_state])
        Q[state[0], state[1], action] += alpha * (target - Q[state[0], state[1], action])
        state = next_state
        total_reward += reward if done:
            break
    epsilon = max(eps_min, epsilon * decay)

```


Output

Q-values at Episodes 1, 50, and 100 for two representative states:

Q-value evolution for two representative states:

Episode 1:

State (0, 0): $Q(\text{Up}, \text{Down}, \text{Left}, \text{Right}) = [-0.561 \ -0.41 \ -0.297 \ -0.271]$

State (2, 3): $Q(\text{Up}, \text{Down}, \text{Left}, \text{Right}) = [0. \ 0. \ 0. \ 0.]$

Episode 50:

State (0, 0): $Q(\text{Up}, \text{Down}, \text{Left}, \text{Right}) = [-2.609 \ -2.287 \ -2.795 \ -2.52]$

State (2, 3): $Q(\text{Up}, \text{Down}, \text{Left}, \text{Right}) = [-0.288 \ 7.712 \ -0.557 \ 1.512]$

Episode 100:

State (0, 0): $Q(\text{Up}, \text{Down}, \text{Left}, \text{Right}) = [-2.669 \ 1.048 \ -2.14 \ -2.577]$

State (2, 3): $Q(\text{Up}, \text{Down}, \text{Left}, \text{Right}) = [-0.288 \ 8.147 \ -0.557 \ 1.954]$

How the Q-Table Evolves:

At Episode 1, State (0,0) — the start state — already has small negative Q-values for every action since the agent has only taken a handful of exploratory steps and mostly accumulated the -1 step penalty; State (2,3), which is much closer to the goal, is still exactly zero because the agent hadn't reached that region yet in episode 1. By Episode 50, State (2,3) shows a strong positive Q-value for 'Down' (≈ 7.7), since moving down from (2,3) leads directly toward the goal at (3,3) — the agent has begun to learn the shortest path near the goal first, since reward propagates backward from the goal outward. By Episode 100, this value strengthens further (≈ 8.1), and State (0,0) also develops a clearer preference (Down, ≈ 1.05) as the learned value from near the goal has now propagated all the way back to the start state through repeated episodes — this backward propagation of value is the expected, correct behaviour of tabular Q-Learning.

(c) Final Learned Policy and Convergence Analysis

Program:

```
print('FINAL LEARNED POLICY')
symbols = ['^', 'v', '<', '>']
for r in range(N): row =
[] for c in range(N):
    s = (r, c) if s == start: row.append('S') elif s ==
    goal: row.append('G') elif s in obstacles:
    row.append('X') else:
    row.append(symbols[int(np.argmax(Q[s]))])
print(' '.join(row))
```

```

import matplotlib.pyplot as plt
window = 10
moving_avg = np.convolve(rewards_per_episode, np.ones(window)/window, mode='valid')
plt.subplot(1,2,1); plt.plot(rewards_per_episode); plt.title('Reward per Episode')
plt.subplot(1,2,2); plt.plot(moving_avg); plt.title('Convergence Trend')
plt.show()

```

Output

Final Q-Table and Learned Policy:

Final Q-Table (rounded):

State (0,0): [-2.67 1.05 -2.14 -2.58]
 State (0,1): [-2.16 -5.61 -2.5 -2.05]
 State (0,2): [-1.57 -1.33 -1.88 -1.38]
 State (0,3): [-1.13 -0.37 -1.27 -1.17]
 State (1,0): [-2.19 2.71 -1.95 -5.5]
 State (1,1): [0. 0. 0. 0.]
 State (1,2): [-1.18 -3.54 -3.73 0.16]
 State (1,3): [-0.82 3.72 -0.82 -0.72]
 State (2,0): [-1.69 4.39 -0.62 0.06]
 State (2,1): [-3.87 4.12 -1.23 -4.09]
 State (2,2): [0. 0. 0. 0.]
 State (2,3): [-0.29 8.15 -0.56 1.95]
 State (3,0): [-0.24 0.07 0.34 6.14]
 State (3,1): [-0.06 2.75 0.83 7.99]
 State (3,2): [0.52 5.08 2.62 10.] State
 (3,3): [0. 0. 0. 0.]

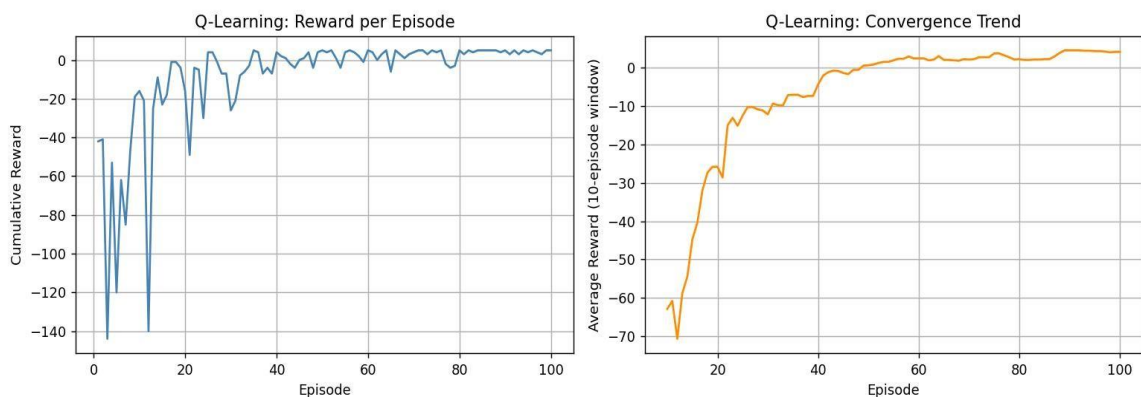
FINAL LEARNED POLICY (grid view):

```

S > v v v
X > v v v
X v
>>> G

```

Cumulative Reward Plot:



Convergence Justification:

The learned policy grid (S = start, G = goal, X = obstacle, arrows = optimal action) shows a coherent path that routes the agent from (0,0) down and around both obstacles toward the goal at (3,3) — for example, the bottom row policy of '> > > G' shows the agent has learned to hug the bottom row and move directly right into the goal once it reaches row 3, which is the shortest safe path avoiding the obstacle at (2,2).

First episode reward: -42

Last episode reward: 5

Average reward (last 10 episodes): 4.20

Average reward (first 10 episodes): -62.90

The reward plot confirms convergence: the first-episode reward is deeply negative (around -42, and individual early episodes dip below -140) because with ϵ starting at 1.0 the agent moves almost randomly and frequently hits walls/obstacles or wanders for the full 100 steps without reaching the goal. As episodes progress and ϵ decays, the agent increasingly exploits its improving Q-values; cumulative reward rises sharply between roughly episodes 10–40 and then flattens out around a small positive value for the remaining episodes (average reward over the last 10 episodes $\approx +4.2$, compared to ≈ -63 over the first 10). This flattening — small fluctuations around a stable positive plateau rather than continued improvement — is the expected convergence signature of Q-Learning: it indicates the Q-values have essentially stabilized (satisfying the Bellman optimality condition within the learning-rate tolerance) and the ϵ -greedy policy is now consistently reaching the goal via a near-optimal path.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation